

National University of Computer and Emerging Sciences

VeriFact

An AI-Powered Fact-Checking System

Group Members

Om · 24K-0711

Vishal · 24K-0625

Zia · 24K-0817

FAST School of Computing

National University of Computer and Emerging Sciences

Karachi, Pakistan

May 2025

1. Introduction

VeriFact is an AI-powered, command-line fact-checking system designed to automatically verify or refute textual claims using real-time web evidence and large language model reasoning. The project was conceived to address the growing problem of misinformation on the internet, where false or misleading statements spread rapidly across social media and online platforms.

Development of VeriFact began in January 2025, with the system reaching a stable v2.0 release by May 2025. The project is being developed by three undergraduate students at the FAST School of Computing, NUCES Karachi, under the supervision of their course instructor. The intended users include journalists, researchers, students, and any individual seeking an objective, evidence-based assessment of a factual claim.

This report covers the following aspects of the project:

- Project purpose, scope, and motivation
- System features and capabilities
- Technology stack and third-party libraries
- Code snippets illustrating the core pipeline
- Pipeline performance and output metrics
- Conclusions and future work

2. Features

VeriFact exposes a rich set of capabilities designed for accuracy, transparency, and usability:

2.1 Multi-Query Evidence Retrieval

Given a user-supplied claim, VeriFact instructs a large language model to generate five semantically diverse search queries targeting different angles of the claim (direct verification, expert opinion, historical context, statistical evidence, and counter-evidence). Each query is issued to DuckDuckGo, retrieving up to 8 results per query for a maximum of 40 raw snippets.

2.2 TF-IDF Clustering

Raw search results are de-duplicated and clustered using K-Means with TF-IDF vectorization. This ensures that evidence from multiple distinct semantic clusters is represented in the final verdict, preventing over-indexing on a single narrative.

2.3 LLM-Based Relevance & Stance Scoring

Each snippet is scored on two dimensions by the LLM: relevance (0–10, how directly it addresses the claim) and stance (–10 to +10, whether it supports or refutes the claim). Scores are boosted by

a source-credibility multiplier derived from a curated tier list of high- and medium-credibility domains.

2.4 Knapsack-Based Snippet Optimization

A 0/1 knapsack solver (Google OR-Tools) selects the optimal subset of snippets that maximises total relevance-weighted score within a character budget. This ensures the most informative evidence is forwarded to the verdict stage without exceeding context-window limits.

2.5 Minimax Debate Engine

The optimized snippets are split into supporting, refuting, and neutral partitions. A structured debate transcript is produced in which the supporting evidence argues for the claim and the refuting evidence argues against it. This adversarial framing helps the verdict model reason about conflicting evidence objectively.

2.6 Verdict Synthesis

The LLM synthesizes the debate transcript into a structured JSON verdict containing: a chain-of-thought reasoning paragraph, a categorical verdict (VERIFIED / FALSE / PARTIALLY TRUE / INCONCLUSIVE), a confidence level (HIGH / MEDIUM / LOW), a plain-language summary, three key findings with source attribution, and any important caveats.

2.7 Interactive Debate Mode

Users may invoke 'debate <topic>' to enter a turn-based adversarial debate with the AI. The system retrieves fresh evidence for every user argument and responds with a citation-backed counterargument, allowing exploration of complex or controversial claims through dialogue.

2.8 Source Credibility Tier System

VeriFact maintains a curated list of high-credibility (Reuters, BBC, Nature, WHO, CDC, NASA, Snopes, PolitiFact, etc.) and medium-credibility (Wikipedia, NPR, Bloomberg, Forbes, etc.) domains. Results from these sources receive a score multiplier of 1.4× and 1.1× respectively.

3. Technology Used

Technology	Version / Model	Purpose
Python 3.11+	3.11	Core programming language
Groq API	llama-3.3-70b-versatile	LLM inference (fast, free-tier)

Technology	Version / Model	Purpose
OpenAI SDK (Groq-compatible)	openai 1.x	HTTP client for Groq endpoint
DuckDuckGo Search	duckduckgo-search	Web search (no API key required)
scikit-learn	TfidfVectorizer + KMeans	Text vectorization & clustering
Google OR-Tools	ortools knapsack_solver	Snippet optimization
Python stdlib	json, textwrap, logging, etc.	Utilities & CLI formatting

4. Code Snippets

4.1 Query Generation

The following function calls the Groq LLM to produce five diversified search queries from the user's claim:

```

QUERY_PROMPT = f"""You are an expert research strategist for a fact-
checking system.
Given a claim, generate exactly {NUM_QUERIES} diverse web-search
queries...
Return ONLY a valid JSON object: {"queries": [{"..."}, ...]}"""

def generate_queries(claim: str) -> list[str]:
    raw = _groq_call(QUERY_PROMPT, f'Claim: "{claim}"',
MAX_TOKENS_QUERIES)
    data = json.loads(raw)
    queries = data.get('queries', [])
    return [q.strip() for q in queries if isinstance(q,
str)][[:NUM_QUERIES]]

```

4.2 Source Credibility Scoring

Credibility multipliers are applied per domain during the scoring phase:

```

CREDIBILITY_TIER = {
    'high': ['reuters.com', 'apnews.com', 'bbc.com', 'nature.com',
            'snopes.com', 'factcheck.org', 'politifact.com', ...],
    'medium': ['wikipedia.org', 'britannica.com', 'npr.org', 'forbes.com',
...],
}

def _domain_credibility(url: str) -> float:
    host = urlparse(url).netloc.lower().lstrip('www.')
    for domain in CREDIBILITY_TIER['high']:
        if host == domain or host.endswith('.' + domain):

```

```

        return 1.4    # high-tier boost
    for domain in CREDIBILITY_TIER['medium']:
        if host == domain or host.endswith('.' + domain):
            return 1.1    # medium-tier boost
    return 1.0          # unknown source

```

4.3 Knapsack Optimization

OR-Tools selects the highest-value snippets within a character budget:

```

def optimize_snippets_knapsack(snippets, max_chars=6000):
    solver = knapsack_solver.KnapsackSolver(...)
    values = [max(1, int(s['score'] * 100)) for s in snippets]
    weights = [[max(1, len(s['snippet'])) for s in snippets]]
    caps = [max_chars]
    solver.init(values, weights, caps)
    solver.solve()
    return [s for i, s in enumerate(snippets) if
            solver.best_solution_contains(i)]

```

4.4 Full Fact-Check Pipeline

The top-level `fact_check()` function orchestrates all stages:

```

def fact_check(claim: str) -> dict:
    queries    = generate_queries(claim)           # Step 1: query gen
    raw        = web_search(queries)              # Step 2: web search
    clustered  = cluster_results(raw)             # Step 3: TF-IDF + K-
Means
    scored     = score_snippets_llm(claim, clustered) # Step 4: LLM
scoring
    optimized  = optimize_snippets_knapsack(scored) # Step 5:
knapsack
    debate, cited, summary = minimax_debate(optimized) # Step 6: debate
    verdict    = synthesize_verdict(claim, debate, cited, summary) #
Step 7
    return verdict

```

5. Pipeline Flow

The diagram below illustrates the end-to-end processing pipeline of VeriFact:

Step 1 — Query Generation

LLM generates 5 diverse search queries from the claim

Step 2 — Web Search

DuckDuckGo retrieves up to 40 raw snippets

Step 3 — TF-IDF Clustering

K-Means clusters results to ensure semantic diversity

Step 4 — LLM Scoring

Each snippet scored on relevance (0–10) and stance (–10 to +10) with credibility boost

Step 5 — Knapsack Optimization

OR-Tools selects maximum-value snippets within 6 000-char budget

Step 6 — Minimax Debate

Evidence is partitioned into supporting / refuting / neutral and debated

Step 7 — Verdict Synthesis

LLM issues VERIFIED / FALSE / PARTIALLY TRUE / INCONCLUSIVE with confidence & reasoning

Verdict categories and their colour-coded console output:

Verdict	Confidence	Meaning
☐ VERIFIED	HIGH / MEDIUM	Credible evidence clearly supports the claim
☐ FALSE	HIGH / MEDIUM	Credible evidence clearly contradicts the claim
☐ PARTIALLY TRUE	MEDIUM / LOW	Evidence supports some aspects while refuting others
☐ INCONCLUSIVE	LOW	Insufficient or too contradictory evidence to decide

6. Conclusion

VeriFact v2 is a fully functional, evidence-driven fact-checking system that combines multiple AI and algorithmic techniques to deliver transparent, cited verdicts on user-supplied claims. The system is currently stable and operational, meeting its initial design goals of multi-source evidence retrieval, adversarial reasoning, and explainable output.

Key accomplishments of the current release include:

- A 7-stage pipeline from claim input to structured JSON verdict
- Credibility-weighted scoring with support for 20+ trusted domains
- Knapsack-based evidence selection ensuring optimal use of the LLM context window
- An interactive debate mode for exploring controversial claims
- Clean, colour-coded CLI output suitable for non-technical users

Planned future enhancements include a web-based graphical interface, support for image and video claim verification, multi-language query generation, integration with premium news APIs for broader coverage, and a persistent claim database with deduplication to accelerate repeated queries. The team is on track for a public beta release by the end of the academic year.

7. References

- [1] Groq Inc. (2024). Groq LPU Inference Engine. <https://groq.com>
- [2] Meta AI. (2024). Llama 3.3 70B Versatile Model Card. <https://ai.meta.com/research/publications/llama-3/>
- [3] DuckDuckGo. (2024). duckduckgo-search Python Library. <https://pypi.org/project/duckduckgo-search/>
- [4] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
- [5] Google. (2024). OR-Tools Knapsack Solver. <https://developers.google.com/optimization/pack/knapsack>
- [6] OpenAI. (2024). OpenAI Python SDK. <https://github.com/openai/openai-python>
- [7] IEEE. (2020). IEEE Citation Reference. <https://ieeeauthorcenter.ieee.org/>